Fiche Git : une issue, une branche, une pull request

À garder ouverte pendant tout le projet. Chaque fois que vous hésitez, suivez les étapes dans l'ordre.

Les 5 règles d'or

1. **On ne code jamais sur `main`.** `main` ne reçoit que des pull requests relues.
2. **Une issue = une branche = une pull request.** Une seule issue à la fois par personne.
3. **On part toujours d'un `main` à jour** (`git pull` avant de créer sa branche).
4. **Des branches courtes** : une issue doit tenir en une ou deux séances. Plus une branche vit longtemps, plus les conflits sont gros.
5. **Max relit et fusionne** chaque pull request.

Nom des branches

```
<prénom>/<numéro de l'issue>-<sujet-court>
```

Exemples : `max/2-serveur-express`, `robin/9-schema-sql`, `hugo/16-charte-css`.

Max	Robin	Hugo
`max/2-serveur-express`	`robin/9-schema-sql`	`hugo/16-charte-css`
`max/3-connexion-postgres`	`robin/10-seed`	`hugo/17-job-board-statique`
`max/4-routes-ads`	`robin/11-crud-companies`	`hugo/18-brancher-api`
`max/5-candidature`	`robin/12-crud-autres-tables`	`hugo/19-connexion-compte`
`max/6-inscription-connexion`	`robin/13-postman`	`hugo/20-style-admin-responsive`
`max/7-middlewares-auth-admin`	`robin/14-page-admin`	
`max/8-auth-me-compte`	`robin/15-readme`	

Ne créez **pas** toutes vos branches à l'avance : une branche se crée au moment où on commence l'issue. Et personne ne crée de branche nommée juste `max`, `robin` ou `hugo` (Git refuserait ensuite les branches `max/...`).

Le cycle complet d'une issue

Exemple avec l'issue #2 de Max. Remplacez le nom de branche et le numéro par les vôtres.

Étape 1 : démarrer l'issue

Sur GitHub, ouvrez l'issue et vérifiez qu'elle vous est assignée. Puis dans le terminal :

```
git switch main                       # revenir sur main
git pull                              # récupérer le travail des autres
npm ci                                # seulement si package.json a changé
git switch -c max/2-serveur-express   # créer la branche ET aller dessus
```

Vérifier : `git branch` affiche une étoile devant `max/2-serveur-express`.

Étape 2 : travailler et faire des petits commits

Un commit = une petite étape qui fonctionne. Plusieurs commits par séance, c'est normal.

```
git status                            # voir ce qui a changé
git add server.js routes/ads.js       # ajouter les fichiers voulus
git commit -m "#2 Ajoute la route de test"
```

- Commencez le message par `#` et le numéro de l'issue : GitHub relie le commit à l'issue.
- Avant chaque `git add`, regardez `git status` : **jamais** de `.env` ni de `node_modules/` dedans.

Étape 3 : envoyer sa branche sur GitHub

```
git push -u origin max/2-serveur-express   # la première fois
git push                                   # les fois suivantes
```

Poussez au moins à la fin de chaque séance : votre travail est sauvegardé et visible par l'équipe.

Étape 4 : se mettre à jour avec `main` avant la pull request

Pendant que vous travailliez, d'autres ont peut-être fusionné dans `main`. On récupère leur travail **dans sa branche**, pour régler les éventuels conflits chez soi et pas sur GitHub.

```
git pull origin main          # fusionne le main de GitHub dans votre branche
npm ci                        # si package.json a changé
# relancez le projet et vérifiez que tout marche encore
git push
```

S'il y a un conflit, voir la section **« Résoudre un conflit »** plus bas.

Étape 5 : ouvrir la pull request

Sur GitHub : bandeau jaune **« Compare & pull request »** (ou onglet *Pull requests* puis *New pull request*).

- **base :** `main`, **compare :** votre branche ;
- **titre :** le titre de l'issue ;
- **description :** ce qui a été fait, comment tester, et `Closes #2` (l'issue se fermera toute seule à la fusion) ;
- **Reviewers :** Max (si c'est la PR de Max, Robin ou Hugo relit) ;
- prévenez l'équipe que la PR est prête.

Si Max demande des corrections : vous corrigez **sur la même branche**, commit, `git push`. La PR se met à jour toute seule, pas besoin d'en créer une nouvelle.

Étape 6 : la fusion (Max)

1. Lire les changements (onglet *Files changed*), avec la checklist de relecture.
2. Récupérer la branche et tester en local si besoin : `git fetch` puis `git switch <branche>`.
3. Bouton **« Merge pull request »**, option **« Create a merge commit »**.
4. Bouton **« Delete branch »** juste après.

Étape 7 : après la fusion, faire le ménage

Chacun, sur son PC :

```
git switch main                      # revenir sur main
git pull                             # récupérer la fusion
git branch -d max/2-serveur-express  # supprimer la branche locale
git fetch --prune                    # oublier les branches supprimées sur GitHub
```

Puis on repart à l'**étape 1** avec l'issue suivante.

Résoudre un conflit

Un conflit veut dire que deux personnes ont modifié **les mêmes lignes**. Git ne sait pas laquelle garder et vous demande de choisir. Ce n'est pas grave.

1. `git status` liste les fichiers en conflit (*both modified*).
2. Ouvrez chaque fichier dans VS Code. Les zones en conflit ressemblent à ça :

   ```
   <<<<<<< HEAD
   votre version
   =======
   la version de main
   >>>>>>> main
   ```

3. Choisissez avec les boutons de VS Code : *Accept Current*, *Accept Incoming* ou *Accept Both*. Ou réécrivez la zone à la main. Il ne doit plus rester aucun `<<<<<<<`, `=======` ou `>>>>>>>`.
4. Testez que le projet marche encore.
5. Terminez :

   ```
   git add fichier-corrigé.js
   git commit -m "#2 Résout le conflit avec main"
   git push
   ```

Si vous ne comprenez pas le conflit, **arrêtez-vous et appelez la personne qui a écrit l'autre version**. Ne supprimez pas son code au hasard.

Les erreurs fréquentes

Problème	Solution
J'ai codé sur `main` sans faire de branche (pas encore commité)	`git switch -c <prénom>/<n°>-<sujet>` : vos modifications vous suivent sur la nouvelle branche.
J'ai commité sur `main` (pas encore poussé)	Prévenez Max **avant de pousser**. Il vous aide à déplacer le commit sur une branche.
`git switch` refuse de changer de branche	Vous avez des modifications non commitées : faites un commit, ou mettez-les de côté avec `git stash` (et `git stash pop` pour les récupérer).
`git push` est refusé (*rejected, fetch first*)	Quelqu'un a poussé avant vous sur cette branche : `git pull`, puis `git push`.
`git branch -d` refuse de supprimer	La branche n'est pas fusionnée. Vérifiez sur GitHub que la PR est bien *Merged* avant toute autre chose.
Après un `pull`, le serveur plante au démarrage	Quelqu'un a ajouté une librairie : `npm ci`.
`.env` apparaît dans `git status`	Ne l'ajoutez pas. Vérifiez que `.gitignore` est à jour sur votre branche (`git pull origin main`).

Les commandes interdites

- `git push --force` : écrase le travail des autres sur GitHub.
- `git reset --hard` : efface vos modifications non commitées, sans retour possible.
- `git add .` sans avoir lu `git status` avant.
- Commiter ou pousser directement sur `main`.

En cas de doute, **on s'arrête et on demande** avant de taper la commande.

Résumé sur une ligne

```
switch main → pull → switch -c branche → code + commits → push → pull origin main → PR "Closes #N" → merge → switch main → pull → branch -d
```